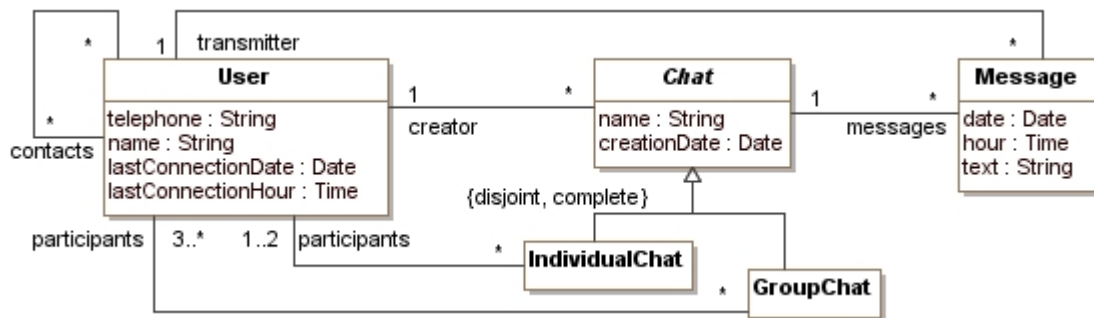


Análisis y Diseño con Patrones

PEC 2: Patrones de Diseño y Asignación de Responsabilidades

Supongamos que estamos desarrollando un software de mensajería electrónica y que disponemos del siguiente diagrama de clases:



Como podemos ver, un usuario tiene un teléfono que lo identifica, un nombre, fecha y hora de la última conexión. Además, un usuario tiene otros usuarios como contactos y es el creador de un conjunto de chats. Cada chat tiene un nombre y una fecha de creación y tiene un conjunto de mensajes. Un chat se identifica por su nombre y el teléfono de su creador. Los chats pueden ser individuales o grupales. Los individuales tienen como mucho dos participantes y los grupales tienen como mínimo 3 participantes. El creador del chat debe ser uno de sus participantes. Los otros participantes del chat deben ser contactos del usuario creador. Los mensajes de un chat disponen de una fecha, hora, texto y tienen un emisor. Un mensaje se identifica por el identificador del chat (nombre del chat y teléfono de su creador) y la fecha y hora del mensaje. Los emisores de los mensajes deben ser participantes del chat. La fecha de un mensaje debe ser posterior a la fecha de creación del chat.

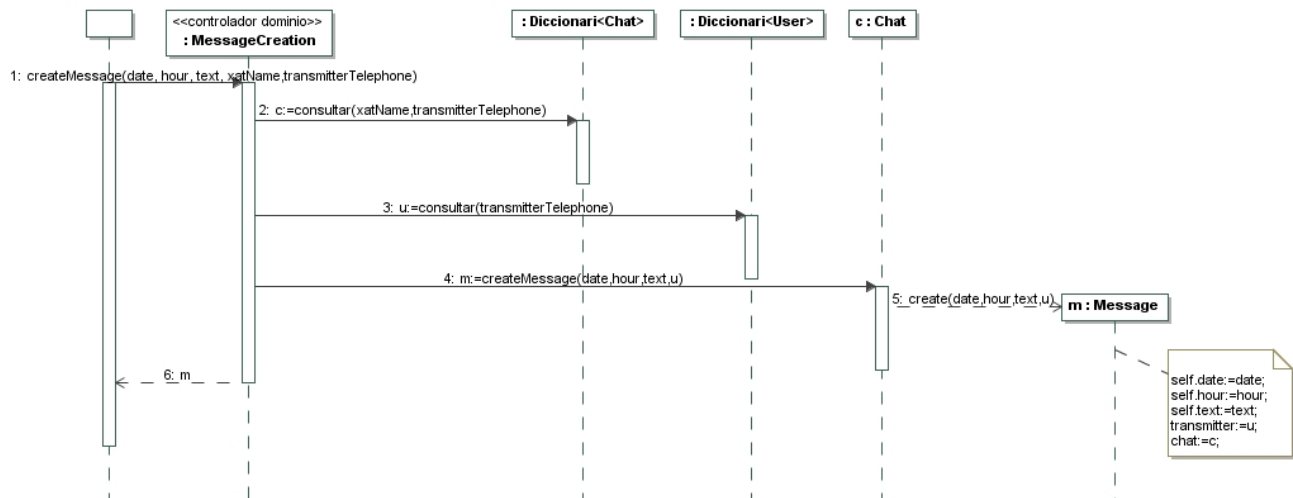
Pregunta 1 (25%)

Supongamos que nuestro sistema debe ser capaz de crear mensajes en los diferentes chats. Para crear estos mensajes es necesario disponer de la información de la fecha, hora, texto y emisor del mensaje, así como del chat donde se publicará el mensaje. Se pide:

- a) ¿Qué patrones de diseño utilizarías en este caso ?
- b) Explica cuál será la clase responsable de crear los mensajes y justifica tu respuesta (pueden existir diferentes clases responsables pero sólo tenéis que proporcionar una).
- c) Muestra el diagrama de secuencia o el pseudocódigo de la operación que crea un mensaje, inicializa su fecha, hora y texto y lo asocia a su emisor y a su chat. Usad los diccionarios (módulo 3, secciones 5.2.2 y 5.2.3) para recuperar instancias a partir de su identificador.

Solución

- a) Utilizaremos el patrón Creador ya que es necesario crear instancias de la clase `Message`.
- b) Asignamos a la clase `Chat` la responsabilidad de crear un mensaje ya que esta clase registra los objetos mensaje. Podríamos también asignar esta responsabilidad a la clase `User` ya que también registra los objetos mensaje o al controlador `MessageCreation` ya que tiene los datos de inicialización que se pasarán al mensaje cuando éste sea creado.
- c) El diagrama de secuencia muestra cómo el controlador obtiene de los diccionarios los objetos chat y usuario. El objeto chat es utilizado para invocar a la operación `createMessage` y el objeto usuario se pasa como parámetro a esta operación. Desde la operación `createMessage` se invoca a la constructora de `Message`. Esta constructora inicializa los atributos del objeto creado así como la asociación con el usuario y con el chat.



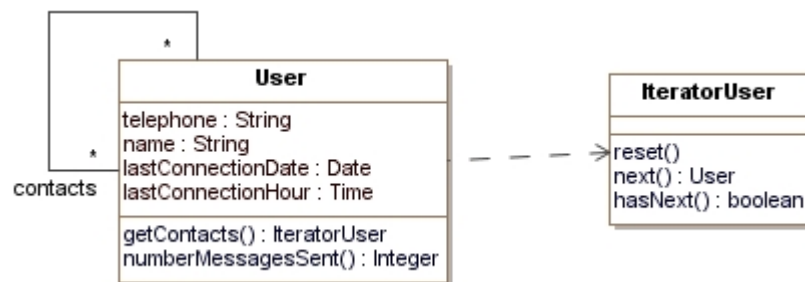
Pregunta 2 (25%)

Supongamos ahora que queremos definir una operación *mostActiveContact ()*: *User* de la clase *User* que devuelve el contacto del usuario *self* (sobre el que se invoca la operación) que ha sido emisor de más mensajes. Se pide:

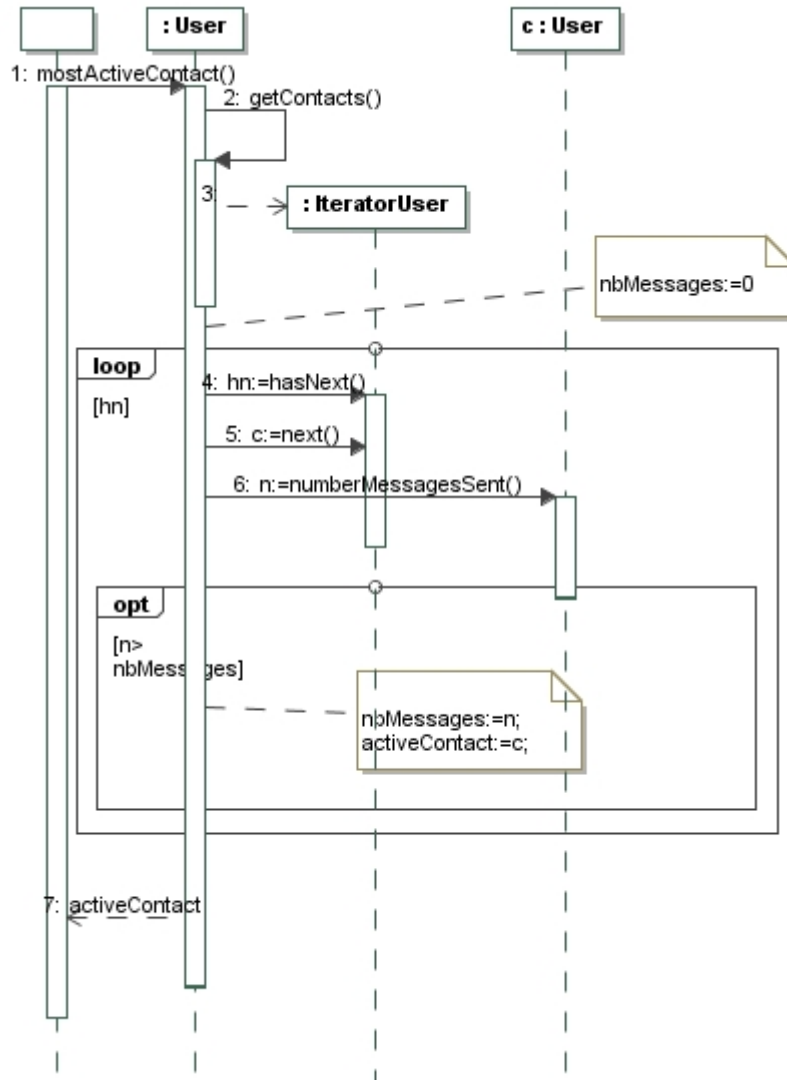
- ¿Qué patrones de diseño utilizarías para diseñar esta operación? Justifica tu respuesta.
- Muestra la parte del diagrama de clases que se ve modificada como resultado de la aplicación de los patrones utilizados.
- Muestra el diagrama de secuencia o el pseudocódigo de la operación *mostActiveContact ()*: *User* de la clase *User*.

Solución

- Utilizaremos el patrón iterador que permite recorrer los contactos de un usuario encapsulando la estructura interna de esta información.
- El diagrama muestra la clase *User* y el iterador de los usuarios para recorrer los contactos del usuario.



- c) El diagrama de sequència mostra el recorrida de los contactos del usuario mediante el iterador. Para cada contacto obtenido en el recorrido se pide el número de mensajes enviados y se encuentra el contacto que tiene más mensajes enviados.



Pregunta 3 (25%)

Queremos añadir dos funcionalidades al software que estamos desarrollando. Estas funcionalidades deben permitir a los usuarios activar e inactivar sus chats. La funcionalidad que inactiva un chat registra la fecha de inactivación y no tendrá ningún efecto si el chat está inactivo. La funcionalidad que activa un chat registra la fecha de activación y no tendrá ningún efecto si el chat está activo. Cuando se crea un chat se crea en estado activo con la fecha de activación igual

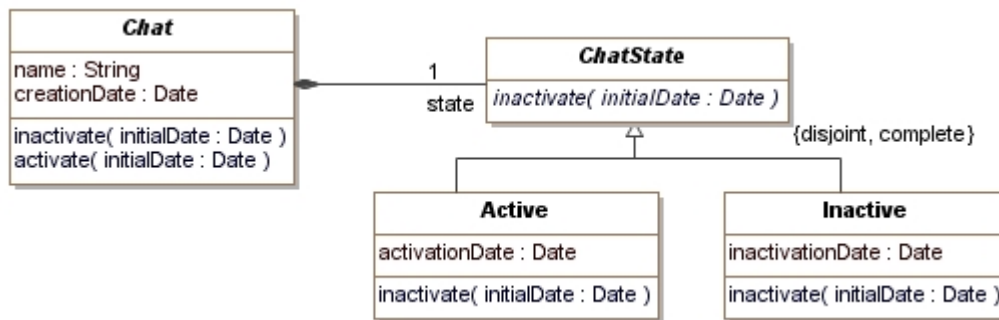
que la fecha de creación. El comportamiento de un chat inactivo es diferente del comportamiento de un chat activo. Por ejemplo, un chat inactivo no puede grabar mensajes.

En concreto, queremos añadir una operación en la clase *Chat* llamada *inactivate* (*initialDate: Date*) que implemente la funcionalidad de inactivación mencionada. No es necesario que implementeis la funcionalidad de activación.

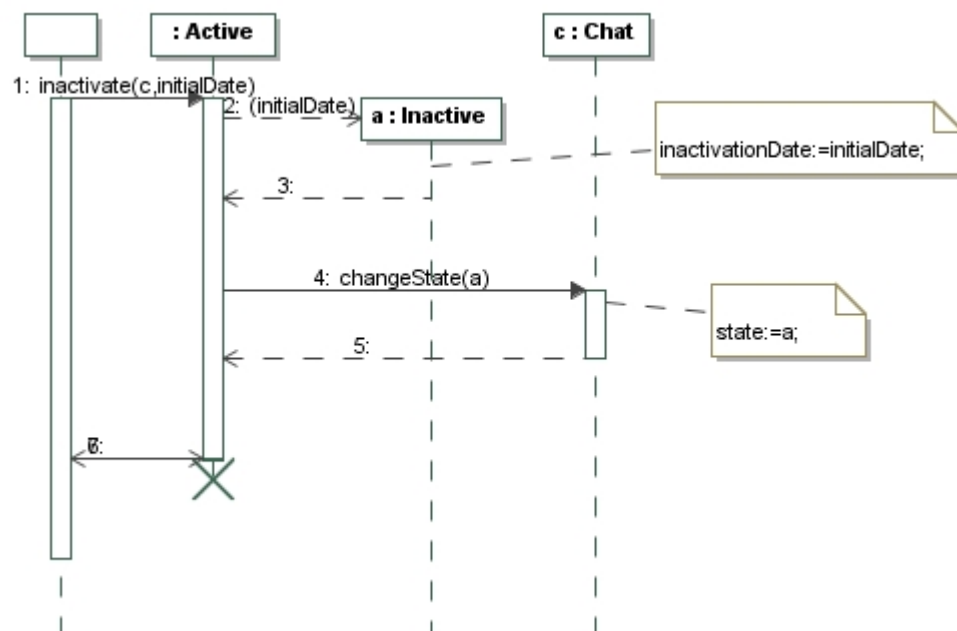
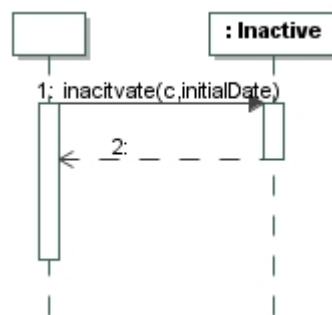
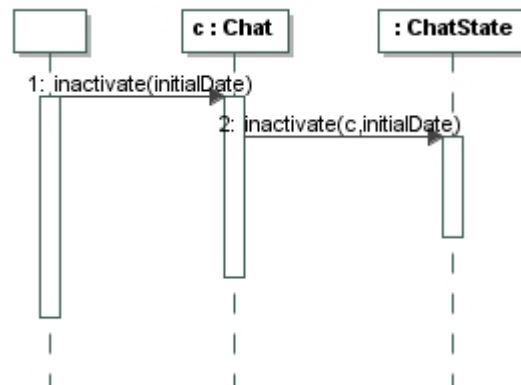
- ¿Qué patrón de diseño recomendarías para este caso (si es que recomiendas alguno)?
- Muestra el diagrama de clases resultante de añadir estas funcionalidades.
- Muestra el diagrama de secuencia o el pseudocódigo de la operación *inactivate* de la clase *Chat*.

Solución

- Como dice el enunciado el comportamiento de un chat inactivo es diferente del comportamiento de un chat activo. Por tanto, el patrón que aplicaremos es el patrón *Estado*.
- El diagrama de clases muestra la aplicación del patrón *Estado* a la clase *Chat*. Se han definido dos estados, el estado activo y el estado inactivo



- El diagrama de secuencia muestra la definición de la operación *inactivate* de la clase *Chat*. Esta operación invoca a otra del mismo nombre al objeto *ChatState* que está asociado con el chat. Esta última operación es abstracta y se define de forma diferente a sus subclasses (estado Activo y Inactivo)



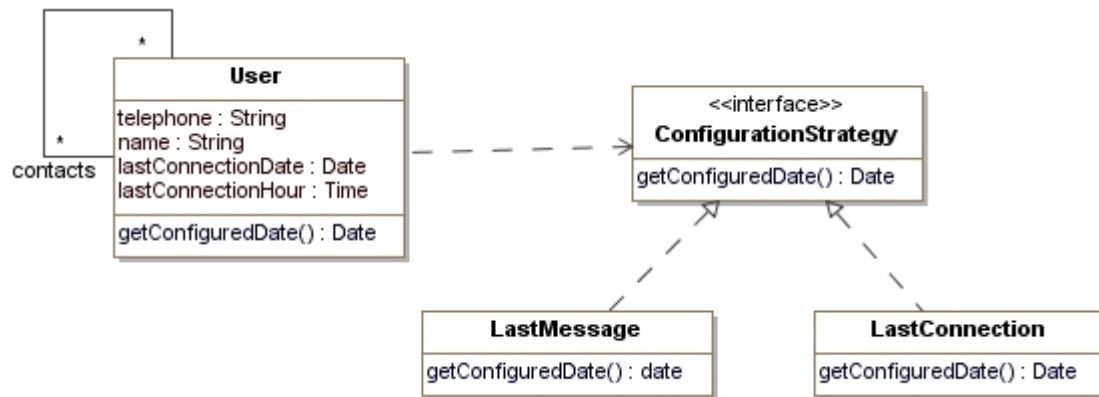
Pregunta 4 (25%)

Supongamos ahora que queremos permitir a los usuarios que puedan configurar parte de la información que se les mostrará cuando entren al sistema de mensajería. En concreto, la pantalla de inicio de este sistema mostrará al usuario todos los chats y, además, mostrará o bien la fecha del último mensaje recibido por el usuario en cualquiera de sus chats o bien la fecha de la última conexión del usuario. El usuario podrá configurar qué fecha quiere que se muestre. Para ello, cada usuario deberá indicar la configuración que desea cuando se dé de alta en el sistema y la podrá cambiar siempre que quiera en tiempo de ejecución. Además, está previsto que en el futuro se puedan definir otras opciones de configuración que puedan retornar otras fechas. Para poder mostrar la información relacionada con las configuraciones iniciales nos piden diseñar la operación *getConfiguredDate ()*: *Date* de la clase *User*. Esta operación devolverá la fecha según la configuración que tenga asociada al usuario. Se pide:

- ¿Qué patrón de diseño recomendarías para este caso (si es que recomiendas alguno)?
- Muestra el diagrama de clases resultante de añadir esta funcionalidad.
- Muestra el diagrama de secuencia o el pseudocódigo de la operación *getConfiguredDate* de la clase *User*. Podéis suponer la existencia de la operación (no será necesario proporcionar el diagrama de secuencia) *User :: getMessageLastDate ()*: *date* que devuelve la fecha del mensaje más reciente recibido por el usuario

Solución

- Aplicaremos el patrón *Estrategia*, ya que nos permite definir una familia de algoritmos (en nuestro caso las operaciones que devuelven las fechas según la configuración seleccionada por el usuario) y permite cambiar la configuración en tiempo de ejecución.
- El diagrama de clases muestra la aplicación del patrón estrategia para representar las dos estrategias (*LastMessage* y *LastConnection*) que permiten mostrar la fecha según la estrategia asociada al usuario.



- c) El diagrama de sequència mostra la definició de la operació *getConfiguredDate* de la classe *User*. Esta operació invoca a otra del mismo nombre de la interfície *ConfigurationStrategy* que està associada con el usuari. Esta última operació se define de forma diferente a las clases que implementan la interfaz.

